

Configurable Static Type Checking for Forth

Jaanus Pöial
Tallinn University of Technology

Abstract

Forth stack comments document the cells consumed and produced by a word. Ordinary comments are not checked and do not record how stack words move or copy individual cells. This paper presents a configurable source-level static type checker implemented in gforth. It describes each word by a typed input/output stack contract, called a stack effect. A validation profile defines the type hierarchy and primitive word effects (specifications). Symbolic indices record the flow introduced by stack shuffling words such as `DUP`, `SWAP`, and `OVER`.

The analysis uses three algebraic operations. Composition handles linear word sequences. Meet operation unifies alternative branches. A finite one-pass-versus-two-pass comparison handles repetition. Forth examples provided in the paper illustrate these operations. They cover type refinement, cell identity, branch merging, reusable inferred effects, and local loop stability. The prototype derives every reported effect without executing concrete values. Its conclusions depend on the user-defined type hierarchy, primitive specifications and source text.

Keywords: Forth; static type checking; stack effects; symbolic execution.

1 Introduction

A Forth word is a named operation. Here, a cell is one single-cell stack item. The data stack is the ordered sequence of those items. A word normally receives operands from that stack and returns its results to the same stack. The conventional notation

```
+   ( n1 n2 -- sum )
```

places the top of stack on the right and says that `+` consumes two numbers and produces one. Such comments are valuable interfaces, but a Forth system usually consumes them as source text rather than treating them as proof obligations. A comment can therefore be missing, stale, or semantically stronger than the checked definition.

Consider the familiar definition

```
: SQUARED  ( n -- n^2 )  DUP * ;
```

The checker can derive `( n - n )` from the declarations of `DUP` and `*`. It can verify the number and identity of the cells used by the phrase. It does not prove the mathematical assertion that the result is the square of the input.

The checker reads three text inputs:

1. a file that defines types, aliases, and ordering relations (type hierarchy);
2. a file that defines word effects and source-reading rules;
3. a Forth source file whose phrases are checked and whose colon definitions are inferred.

A *primitive word* is a word whose effect comes directly from the second file. A *colon definition* begins with `:` and ends with `;`. The word that it defines receives an inferred effect from the source

between those delimiters. A top-level phrase is checked and traced but does not add a dictionary entry. The prototype consumes the three files and maps each known word to one stack effect. At `;`, the inferred body effect is added to this dictionary. The prototype analyzes symbolic stack cells; it does not execute the Forth definition. The inferred definitions are stored in a log file.

The design builds on earlier work on stack effects and symbolic type variables [3, 4, 5, 6]. The following topics are discussed:

- how a later word can constrain an earlier input;
- how two branch effects are combined;
- why an effect displays only the top cells that it touches;
- why the loop check compares one pass with two passes.

Sections 2–4 define the analysis used here. Sections 5–7 derive all twelve test results and the four helper effects.

2 Typed Symbolic Stack Effects

2.1 Types and compatibility

Let T be a finite set of abstract cell types with a precision order $\preceq$. The relation $t \preceq u$ means that t is more specific than u . A value of type t may therefore be used where u is required. This order is the profile’s subtype relation. The Forth-2012-like profile used for the examples contains, among other edges, the transitive chains

$$\begin{aligned} \text{a-addr} &\preceq \text{c-addr} \preceq \text{addr} \preceq \text{u} \preceq \text{n} \preceq \text{x}, \\ \text{char} &\preceq \text{+n} \preceq \text{u} \preceq \text{n} \preceq \text{x}, \quad \text{flag} \preceq \text{x}. \end{aligned}$$

The flag type is deliberately separate from the numeric chain in this profile. Another profile may classify flags as numbers. The hierarchy is therefore a policy for one target vocabulary. It is not a universal claim about Forth cells. Names such as `x`, `X`, and `cell` may be aliases for one type.

Two types are *comparable* when one is below the other in $\preceq$. Comparable types are compatible. Matching them retains the more specific type:

$$\text{match}(t, u) = \begin{cases} t, & t \preceq u, \\ u, & u \preceq t, \\ \text{undefined}, & \text{otherwise.} \end{cases}$$

The prototype does not search for a third common subtype when t and u are incomparable. An undefined match is a type clash.

2.2 Symbols, effects, and the implicit frame

A symbolic cell is a type with an optional positive identity index, written $t[i]$. Equal indices denote the same abstract origin. Different indices do not claim that the concrete values differ. Every unindexed occurrence starts as an independent symbol. An unindexed output is therefore a fresh abstract result. Indices exist only in the checker. They are not addresses or runtime tags.

The declarations of three stack words illustrate the notation:

```
DUP   ( x[1] -- x[1] x[1] )
SWAP  ( x[2] x[1] -- x[1] x[2] )
OVER  ( x[2] x[1] -- x[2] x[1] x[2] )
```

Before each word occurrence is analyzed, its local indices receive fresh numbers. Two primitive calls therefore cannot share an index by accident. After analysis, *normalization* rennumbers the surviving indices from one upward. Renumbering itself changes only their printed names. The prototype also compacts identities: a derived index that records only an implicit one-to-one correlation may be hidden, while an explicitly declared or multiply occurring correlation is retained. Once hidden, that pairwise correlation is unavailable to later composition. This deliberate precision loss keeps stored effects small but means that inferred effects need not be principal.

An effect $e = (L - R)$ contains two finite symbolic stacks, with their tops on the right. A stack *suffix* is the group of top cells displayed by the effect. Deeper cells form an implicit *frame*. For every frame F , the contract acts as

$$F L \mapsto F R.$$

For example, (**a-addr** - x) for @ does not require the entire data stack to contain one cell. It requires only that the top cell be an aligned address; any deeper F passes through unchanged. The identity effect is (-). Write ϵ for the empty stack. A clash is a separate failure result and must not be confused with the identity effect. The set of symbolic effects is the checker's *abstract domain*. An effect operation is *partial* when some arguments produce no effect. In those cases, this checker reports a clash.

3 Main Effect Operations

3.1 Sequential composition

Write typed symbolic cells by juxtaposition, with the top on the right, and let e and f denote possibly empty sequences of such symbols. The two symbolic equations from the presentation capture the basic cases:

$$(a - b) \cdot (eb - d) = (ea - d), \quad (a - fc) \cdot (c - d) = (a - fd).$$

In the first equation, the second effect consumes b but also needs the deeper sequence e ; that sequence therefore becomes part of the composite input. In the second, the first effect leaves f below c ; consuming c leaves f below the new result d . In case of type mismatch the result is zero (type clash).

More generally, $p = (L_p - R_p)$ followed by $q = (L_q - R_q)$ is written $p \cdot q$. Boundary cells are matched from the top inward. Comparable types are replaced by their more specific type, and that refinement is substituted through every occurrence of the same symbolic index. Thus a late @ requirement can propagate through copies made earlier by **SWAP** and **DUP**. Incomparable boundary types make composition undefined. A sequence is composed from left to right, and the empty sequence has the identity effect. This operation extends earlier untyped stack-effect composition [3].

3.2 Common branch contracts

Alternative paths use the common-contract operation $p \sqcap q$. It produces one must-style interface rather than a set of path effects. Its two basic symbolic equations are

$$(a - b) \sqcap (ea - eb) = (ea - eb),$$

$$(ea - eb) \sqcap (ec - ed) = (e \min(a, c) - e \max(b, d)).$$

The first equation makes the implicit frame explicit: $(a - b)$ also acts as $(ea - eb)$, so that is the common contract. In the second equation, $\min(a, c)$ means the more specific of two comparable input types, while $\max(b, d)$ means the more general output type. These are precision-order operations, not numeric minimum and maximum. Inputs narrow because a caller must satisfy both paths; outputs widen because a client may assume only what both paths guarantee.

Positions are aligned from the top. An identity index survives only when both paths preserve the same input-to-output relationship; a path-dependent permutation is forgotten. Unequal displayed depths are valid only when the extra input and output depths are equal, because the omitted prefix must pass through unchanged:

$$|L_p| - |L_q| = |R_p| - |R_q|.$$

A different net stack change or an incomparable aligned type makes the meet undefined (zero).

Write $\llbracket P \rrbracket$ for the inferred effect of a source segment P . The selector is composed separately for a conditional:

$$\llbracket \text{IF } P \text{ ELSE } Q \text{ FI} \rrbracket = (\text{flag} - -) \cdot (\llbracket P \rrbracket \sqcap \llbracket Q \rrbracket).$$

Without ELSE, Q is the identity effect.

3.3 Finite repetition

For a repeated phrase with one-pass effect p , the prototype computes

$$\pi_2(p) = p \sqcap (p \cdot p),$$

provided both the composition and common-contract operations are defined. This one-versus-two comparison is finite and terminating. It detects many bodies that grow, shrink, or change the stack incompatibly on a second pass.

The notation has a narrow meaning. It does not summarize every possible iteration count. One and two passes can agree after information has been lost. A later pass may then remove another correlation. A full loop analysis would propagate the loop-header state until that state stops changing. π_2 does not perform that process. It also proves neither termination nor properties of concrete values. It is only a local stack-stability test.

4 Prototype Inputs

The prototype loads a type profile, a word-and-control profile, and the example source. Each example below introduces its primitive declarations at the point of use rather than repeating them in a separate catalogue.

Control syntax is also profile-driven. A conditional captures its two branches and composes the selector with their common contract. A BEGIN-WHILE-REPEAT form captures a prefix and a body, while BEGIN-UNTIL captures one body including the final selector; the configured recipe applies π_2 to those segments. The profile therefore determines both how source is divided and which symbolic operation combines each part.

With these profiles, the prototype accepts all twelve tests and the four helper definitions. Their inferred effects are derived once, in the following sections. The accepted top-level phrase is discussed with `test12`.

5 Straight-Line Examples

5.1 Test1: stack shape without value evaluation

```
: test1 3 4 5 - + ;
```

Every literal has effect (- n). Naming fresh abstract results a , b , and c , the stack evolves as

$$\epsilon \longrightarrow n_a \longrightarrow n_a n_b \longrightarrow n_a n_b n_c \longrightarrow n_a n_d \longrightarrow n_e.$$

Subtraction consumes n_b, n_c and produces a fresh n_d . Addition consumes n_a, n_d and produces n_e . No input is required. One number is left:

```
test1 ( - - n ).
```

Concrete execution would calculate $4 - 5 = -1$ and then $3 + (-1) = 2$. The checker does not perform that calculation. Its domain contains the type n , but no integer constants or arithmetic expressions. It establishes only (- n).

5.2 Test2: a generic shuffle specialized to flags

```
: test2 OR FALSE SWAP ;
```

OR first requires two flags and leaves one fresh flag. FALSE adds another flag. The generic inputs of SWAP then match these two results. Both inputs are refined from x to flag :

$$\text{flag}_a \text{flag}_b \xrightarrow{\text{OR}} \text{flag}_c \xrightarrow{\text{FALSE}} \text{flag}_c \text{flag}_d \xrightarrow{\text{SWAP}} \text{flag}_d \text{flag}_c.$$

The inputs are consumed by OR; neither output is correlated with an original input. The reported interface therefore needs no visible indices:

```
test2 (flag flag - flag flag).
```

Although one output is produced by the word named FALSE, the checker has no constant-propagation domain and promises only its declared type.

5.3 Test3: backward refinement through SWAP and DUP

```
: test3 SWAP DUP @ ;
```

Let the initial suffix be $A B$, with B topmost. SWAP produces $B A$ and DUP produces $B A A$. The final @ requires the top copy of A to be an **a-addr**. Because both copies carry the same symbolic index, the substitution also refines the untouched copy and the original input:

$$A B \xrightarrow{\text{SWAP}} B A \xrightarrow{\text{DUP}} B A A \xrightarrow{@} B A X_{\text{fresh}}.$$

After normalized numbering, A is **a-addr**[2] and B is $x[1]$. Hence

```
test3 (a-addr[2] x[1]
      - x[1] a-addr[2] x).
```

This is the key identity-flow example. Tracking only stack depth would miss the address requirement. Tracking types without indices would also lose information. Such an analysis could refine only the consumed copy. It could not identify the surviving copy as the same aligned address.

6 Branch Examples

6.1 Test4: an explicit effect versus an implicit frame

```
: test4 IF 1 + FI ;
```

The true branch $1 +$ pushes an n , then needs one additional n below it for $+$. Its inferred effect is $(n - n)$. The absent **ELSE** branch has the identity effect $(-)$. These displayed depths differ, but the empty branch preserves an arbitrary deeper frame. Making one frame cell explicit gives the alignment

$$\begin{array}{ll} \text{true} & : (n - n), \\ \text{false} & : (\underline{n} - -\underline{n}). \end{array}$$

Both paths preserve a one-cell numeric stack shape. They do not preserve the same value. The false path returns the original number. The true path returns the result of addition. No output identity is claimed. After the leading **IF** consumes its selector, the complete definition is

$$\text{test4 } (n \text{ flag} - n).$$

This example explains why comparing only the explicitly printed sides would be too strict. Stack effects denote transformations under an implicit frame.

6.2 Test5: a common prefix survives, a branch-dependent identity does not

```
: test5 IF DUP ELSE 0 FI ;
```

The true branch is $(x[1] - x[1] \ x[1])$. The false branch explicitly has $(- \ n)$, but it leaves the deeper frame unchanged. Exposing one frame cell aligns the branches as follows:

$$\begin{array}{ll} \text{true} & : (x[1] - x[1] \ x[1]), \\ \text{false} & : (x[1] - x[1] \ n). \end{array}$$

The lower output is the original input on both paths, so its index survives. The top output is the same input on the true path but a fresh numeric literal on the false path. Its path-independent type is the more general type x . It cannot retain the input index. The branch contract is thus

$$(x[1] - x[1] \ x).$$

Composing the selector yields

$$\text{test5 } (x[1] \text{ flag} - x[1] \ x).$$

The result loses two facts. First, n is widened to x at the differing output. Second, a correlation valid on only one path is discarded.

6.3 Test6: compatible shape with no common permutation

```
: test6 IF ROT ELSE @ FI ;
```

Write the pre-branch suffix as $A \ B \ C$, with C topmost. The true branch rotates it:

$$A \ B \ C \mapsto B \ C \ A.$$

The false branch $@$ displays only one input and one output. Its implicit two-cell frame makes the comparable transformation

$$A \ B \ C \mapsto A \ B \ D,$$

where C must be **a-addr** and D is a fresh **x**. At the input, ROT's generic requirement and @'s address requirement meet at the more specific **a-addr**. The input contract is consequently $(\text{ x x a-addr - ... })$.

The output positions have the path pairs (B, A) , (C, B) , and (A, D) . None of these establishes one common source identity. Their only common output type is **x**, so the branch contract is

$$(\text{ x x a-addr - x x x }).$$

After selector consumption, the inferred definition is

$$\text{test6}(\text{ x x a-addr flag - x x x }).$$

The checker accepts the equal net stack change. It forgets which input reaches which output. An analysis that stores each path separately could retain both permutations. This checker stores one common contract.

6.4 Test9: input and output variance in isolation

```
: test9 IF @ ELSE C@ FI ;
```

The two branch effects are

$$(\text{ a-addr - x }) \quad \text{and} \quad (\text{ c-addr - char }).$$

An input to the whole conditional must satisfy both address requirements. Since $\text{a-addr} \preceq \text{c-addr}$, the common input is the more specific **a-addr**. An output client, however, needs a guarantee valid for either **x** or **char**. Since $\text{char} \preceq \text{x}$, that guarantee is the more general **x**. Thus

$$(\text{ a-addr - x }) \sqcap (\text{ c-addr - char }) = (\text{ a-addr - x }),$$

and selector composition gives

$$\text{test9}(\text{ a-addr flag - x }).$$

This small example is the clearest demonstration of contract variance: branch inputs are narrowed, whereas outputs are widened.

6.5 Test10: merging effects inferred for helper words

```
: P INVERT HERE ! DP ! DP @ DP C@ SWAP ;
: Q SWAP ! DP @ DP ;
: test10 IF P ELSE Q FI ;
```

This example first checks two ordinary colon definitions and then uses their inferred effects as branch contracts. For **P**, let the initial suffix be **x flag**. **INVERT HERE !** consumes the new aligned address and the flag result, leaving the lower **x**. **DP !** then consumes that cell and another aligned address. Finally, **DP @** and **DP C@** produce **x char**, which **SWAP** exchanges. Thus

$$\text{P}(\text{ x flag - char x }).$$

Both outputs are fresh results of fetches; neither is an original input.

For **Q**, **SWAP !** consumes an **a-addr** and an arbitrary cell. **DP @ DP** then produces a fresh cell and a fresh aligned address:

$$\text{Q}(\text{ a-addr x - x a-addr }).$$

The common branch input narrows **x** against **a-addr** in the lower position and **flag** against **x** in the upper position. The two output pairs are (**char**, **x**) and (**x**, **a-addr**); each position therefore widens to **x**. No identity is common to the paths:

$$\llbracket P \rrbracket \sqcap \llbracket Q \rrbracket = (\text{a-addr flag} - \text{x x}).$$

After **IF** consumes the selector, the complete result is

$$\text{test10}(\text{a-addr flag flag} - \text{x x}).$$

Unlike **test9**, this case shows that effects inferred for user-defined words are immediately reusable and that variance is applied independently at each aligned position.

7 Repetition Examples

7.1 Test7: one-versus-two exposes both flag requirements

```
: test7 BEGIN SWAP OVER WHILE INVERT REPEAT ;
```

The profile divides this structure into the repeated prefix **SWAP OVER WHILE** and the repeated body **INVERT**. Consider one prefix pass on $A B$:

$$A B \xrightarrow{\text{SWAP}} B A \xrightarrow{\text{OVER}} B A B \xrightarrow{\text{WHILE}} B A.$$

WHILE consumes the duplicate of the original top cell, so B must be a flag. A second prefix pass transforms $B A$ back to $A B$ and tests A . Consequently the two-pass effect requires both original cells to be flags. The common summary preserves two flag-typed positions. It cannot promise one permutation. One pass swaps the cells, while two passes restore them. The summary is therefore

$$(\text{flag flag} - \text{flag flag}).$$

The second repeated region, **INVERT**, has the stable type effect $(\text{flag} - \text{flag})$. Composing it with the prefix summary leaves the same two-cell interface, giving

$$\text{test7}(\text{flag flag} - \text{flag flag}).$$

This is a useful rejection test for changing stack shape and incompatible second-pass types. It is not a proof that the reported identity-free summary characterizes every number of loop iterations or that the loop terminates.

7.2 Test8: a data-stack-neutral loop pass

```
: test8 BEGIN KEY 0= UNTIL ;
```

A complete repeated pass has the data-stack trace

$$\epsilon \xrightarrow{\text{KEY}} \text{char} \xrightarrow{0=} \text{flag} \xrightarrow{\text{UNTIL}} \epsilon.$$

The loaded **x** contract for **0=** accepts **char** directly. The presentation's narrower **n** annotation would also accept it because $\text{char} \preceq \text{n}$. **UNTIL** consumes the resulting flag. One pass has the identity data-stack effect. Two passes have the same effect. Their common contract is

$$\text{test8}(-).$$

This conclusion concerns only the modeled data stack. **KEY** still has an I/O side effect. The result does not promise a key that meets the exit condition. It also does not prove termination.

7.3 Test11: a reusable body refines both loop cells

```
: BODY SWAP DUP @ 0= ;
: test11 BEGIN BODY UNTIL ;
```

The first three words of `BODY` repeat the identity flow from `test3`. The fetch result is a generic `x`; the loaded profile's broad (`x - flag`) declaration for `0=` consumes it and produces a selector. The helper effect is

$$\text{BODY}(\text{a-addr}[2] \ x[1] \ - \ x[1] \ \text{a-addr}[2] \ \text{flag}).$$

`UNTIL` consumes the final flag. One complete pass therefore swaps two cells and requires the first one to be an aligned address:

$$p = (\text{a-addr}[2] \ x[1] \ - \ x[1] \ \text{a-addr}[2]).$$

On a second pass, the other original cell reaches `@` and is refined to `a-addr`. Hence $p \cdot p$ requires two aligned addresses and restores their order. One pass swaps the origins while two passes preserve them, so no input/output identity is common to both paths. Their types are common:

$$p \sqcap (p \cdot p) = (\text{a-addr} \ \text{a-addr} \ - \ \text{a-addr} \ \text{a-addr}).$$

This is the reported effect of `test11`. It combines reusable inferred identity information with the same two-pass refinement principle seen in `test7`.

7.4 Test12: a concrete boundary of one-versus-two

```
: BODY23 ROT INVERT DROP 0 ;
: test12 BEGIN BODY23 KEY? UNTIL ;
BODY23 BODY23 BODY23
```

Let the three inputs of `BODY23` be $A \ B \ C$, with C topmost. `ROT` produces $B \ C \ A$, so `INVERT` requires A to be a flag. `DROP` removes the resulting flag, and the literal produces a fresh `n`. The reusable effect is

$$\text{BODY23}(\text{flag} \ x[2] \ x[1] \ - \ x[2] \ x[1] \ n).$$

In `test12`, `KEY?` produces the flag consumed by `UNTIL`; those two words have no net data-stack effect. Thus one complete loop pass has the same effect as `BODY23`. A second pass requires B to be a flag and maps $A \ B \ C$ to $C \ n \ n$. Meeting one and two passes retains the requirements that A and B are flags, widens the first two output positions to `x`, and retains `n` at the top:

$$\text{test12}(\text{flag} \ \text{flag} \ x \ - \ x \ x \ n).$$

No index survives because the first output originates at B after one pass but at C after two, while the remaining aligned outputs also lack a common origin.

The final top-level phrase deliberately takes a third pass. Its symbolic trace is

$$\text{flag} \ \text{flag} \ \text{flag} \longrightarrow \text{flag} \ \text{flag} \ n \longrightarrow \text{flag} \ n \ n \longrightarrow n \ n \ n.$$

The third pass introduces a requirement not present in the one-versus-two summary: C must also be a flag. This does not contradict the rule, because π_2 was defined as a bounded local comparison, not as an inductive loop invariant. It gives a concrete reason not to read the accepted `test12` contract as valid for every iteration count.

8 Discussion

8.1 What the examples establish

The twelve tests isolate complementary properties of the symbolic domain. `test1` shows that the checker infers shape and cell type without constant evaluation. `test2` shows subtype specialization of a generic stack word. `test3` demonstrates bidirectional information flow: execution order is left to right, but a late requirement propagates backward through shared symbolic identities.

The branch examples show why an effect is more than a pair of stack heights. `test4` depends on the implicit frame. `test5` retains only an identity common to both paths. `test6` loses a branch-dependent permutation. `test9` isolates input/output variance, and `test10` applies that variance to effects inferred for helper words. The loop examples expose both the value and the limit of the finite rule. A second pass refines another cell in `test7`; a balanced pass becomes identity in `test8`; and `test11` carries identity information through a reusable body. Finally, `test12` and the following three-BODY23 phrase show directly why one-versus-two is not an all-iteration invariant.

All reported outputs are inferred from primitive contracts. The illustrative comments in the presentation are not declarations. This rule prevents a stale comment from changing inference. Semantic labels such as `value` or “address contents” are therefore outside the domain. A richer profile would be needed to represent them.

8.2 Configuration as an analysis parameter

The same source may receive a different verdict under another valid profile. For example, one profile may classify `char` as numeric. It accepts the displayed `KEY 0=`. Another profile may keep `char` and `n` unrelated. A numeric-only `0=` then rejects the phrase. Even under the hierarchy used here, replacing the loaded broad `( x - flag )` contract by `( n - flag )` would still accept `test8` but reject `BODY`, because `@` guarantees only `x`. The target Forth system determines which policy is appropriate. The profile makes that policy reviewable.

External control declarations provide similar flexibility. They also enlarge the trusted base. A recipe might omit a selector effect. It might also put `UNTIL` outside its repeated segment. Either error can produce a wrong summary even when the three operations are correct. The configuration should therefore be reviewed with the checked code.

9 Validity and Limitations

Acceptance has a precise but bounded interpretation:

- every checked linear boundary has enough modeled cells of comparable types;
- every checked branch admits one common contract under the defined alignment and variance rules;
- every checked repetition admits the finite $p \sqcap (p \cdot p)$ summary;
- parser roles, delimiters, and compilation state fit the active profile.

It does not imply full Forth program correctness. The principal threats to a stronger interpretation are as follows.

Trusted declarations. Primitive and parser specifications are assumptions. If `@`, `KEY`, or a control word is declared incorrectly, analysis of its clients cannot make the result reliable.

Single effects, compaction, and comparable types. The dictionary stores one effect per word. Branch merging also returns one common contract. It cannot retain a separate permutation for each path. Normalization may discard an implicit one-to-one identity, so a stored derived effect need not retain every correlation available in its construction trace. Aligned types must be directly comparable. The checker does not search for a third common type.

Finite loop approximation. The one-versus-two comparison does not prove a result for every iteration count. A stronger analysis would maintain an abstract stack at the loop header. It would propagate the body until that stack stopped changing. Another design would require a declared loop interface and check every back edge against it [1].

Forth language coverage. Return-stack manipulation, unrestricted EXECUTE, recursion, non-local THROW, native defining words, self-modifying dictionaries, and unrestricted metaprogramming require additional abstractions. The checker should complement runtime tests, code review, and target-system knowledge.

The examples are explanatory cases, not a performance benchmark or a proof that every accepted program is safe. Each case makes one operation visible, and the prototype’s reported effects agree with `examples.txt.log`.

10 Related Work

Forth has long used stack-effect notation as an interface convention. The Forth standard defines data types and control semantics [2]. It does not make each source comment a static judgment. Earlier work developed algebraic composition and multiple effects for control flow [3, 4]. Later work added typed symbolic variables and conservative control-flow operations [5, 6]. The prototype described here also loads parser and control behavior from a profile.

Stoddart and Knaggs also studied type inference for stack languages [8]. Saabas and Uustalu gave compositional type systems for stack-based low-level code [7]. Forth source can additionally contain parser words and dictionary extensions. The profile handles that variability, but it becomes part of the trusted input.

Abstract interpretation offers a stronger treatment of loops [1]. It propagates abstract states until they stop changing. The one-versus-two rule performs only the first bounded comparison. An inferred loop effect must be read with that distinction in mind.

11 Conclusion

Typed symbolic stack effects turn a familiar Forth notation into executable, configurable documentation. Sequential composition checks touching stack boundaries and propagates refined types through shared identities. Branch merging computes one conservative contract, narrowing inputs, widening outputs, and preserving only path-independent correlations. Repetition compares one pass with two to obtain a useful but explicitly finite stability summary.

The tests provided make these mechanisms concrete. Three initial linear definitions show type and identity propagation. Five conditionals show implicit frames, input narrowing, output widening, information loss, and reuse of inferred helper contracts. Four loops show what the bounded repetition rule accepts; the last is paired with a three-pass phrase that exposes exactly what the rule does not establish. The prototype derives every reported effect under the selected profile. It is a practical checker for the configured subset of Forth. It is not a proof of functional behavior or termination. Stronger assurance requires validated primitives, path-sensitive alternatives, less lossy identity handling, and full loop analysis.

References

- [1] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of POPL*, pp. 238–252, 1977. doi:10.1145/512950.512973. Public full text: <https://www.di.ens.fr/~cousot/publications.www/CousotCousot-POPL-77-ACM-p238--252-1977.pdf>.
- [2] Forth 200x Standardisation Committee. *Forth 2012 Standard*. Draft proposed standard, 10 November 2014. Public full text: <https://forth-standard.org/standard/intro>.
- [3] J. Pöial. Algebraic specification of stack effects. *Forth Dimensions*, 16(4):18–20, 1994. Public full text: <https://www.forth.org/fd/FD-V16N4.pdf>.
- [4] J. Pöial. Multiple stack-effects of Forth programs. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 400–406, 1992. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef91/poial.pdf>.
- [5] J. Pöial. Stack effect calculus with typed wildcards, polymorphism and inheritance. Presentation and proceedings abstract, 18th EuroForth Conference, 2002. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef02/poial02.pdf>.
- [6] J. Pöial. Typing tools for typeless stack languages. In *Proceedings of the 22nd EuroForth Conference*, pp. 40–46, 2006. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef06/poial06.pdf>.
- [7] A. Saabas and T. Uustalu. Compositional type systems for stack-based low-level languages. In *Proceedings of CATS 2006*, CRPIT 51, pp. 27–39, 2006. Public full text: https://web.archive.org/web/20120325100058id_/http://crpit.com/confpapers/CRPITV51Saabas.pdf.
- [8] B. Stoddart and P. J. Knaggs. Type inference in stack based languages. In *1991 FORML Conference Proceedings, euroFORML'91*, pp. 407–421, 1991. Public full text: <https://www.complang.tuwien.ac.at/anton/euroforth/ef91/stoddart.pdf>.